Tecnológico de Monterrey

13. Lematización, TF-IDF y similitud del coseno

Ciencia de datos para la toma de decisiones II

Jorge Juvenal Campos Ferreira

juvenal.campos@tec.mx

Hacia donde vamos:

La **lematización** es la parte del **preprocesamiento** en el que trabajamos nuestros textos para poder analizarlos en alguno de los siguientes modelos:

1. **Análisis descriptivos y exploratorios (*frecuencias de palabras, n-gramas, visualizaciones*)**
2. **Análisis de tópicos**
3. **Análisis de sentimientos**
4. **Clasificación y regresión de textos**
5. **Análisis semántico avanzado (*modelos vectoriales de palabras, similitudes semánticas*)**

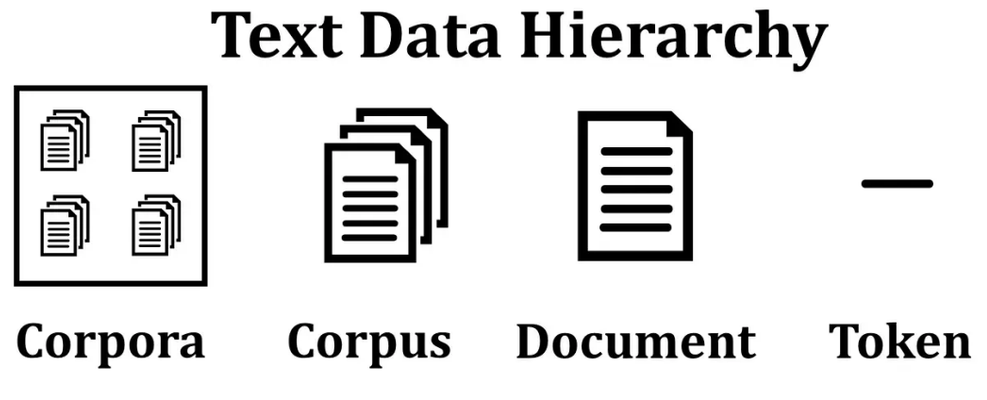
Corpus / Corpora

Un **corpus** (plural: *corpora*) es, en lingüística y procesamiento de lenguaje natural (NLP), una **colección organizada de textos** que se utiliza como base para el análisis del lenguaje.

Es un conjunto de textos: puede contener artículos, libros, entrevistas, discursos, etc.

Suele diseñarse para que sea representativo de un idioma, de un dominio especifico o de un estilo, o que presente los documentos necesarios para realizar una investigación.

Sirven para entrenar modelos de lenguaje, estudiar la frecuencia de palabras y expresiones, detectar patrones o desarrollar aplicaciones.



☆ Lema:

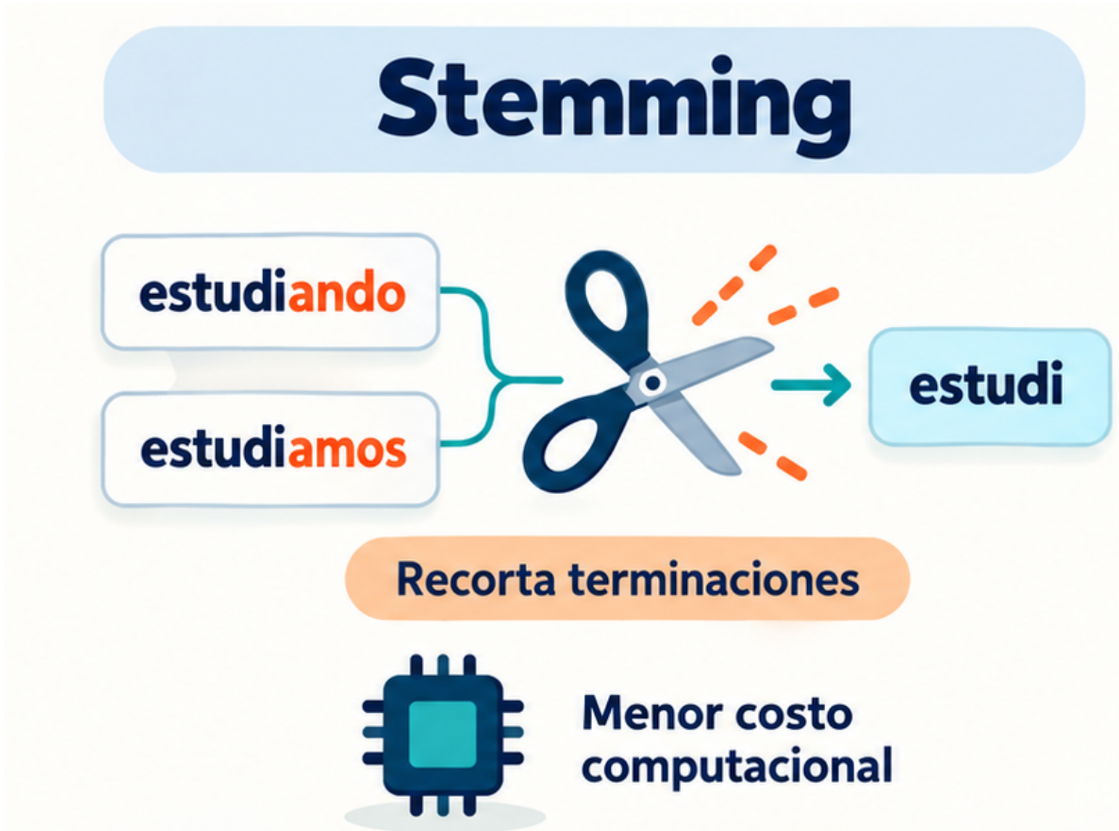
Un **lema** es la **forma base o canónica de una palabra**, es decir, la forma que usarías para buscarla en un diccionario.

En procesamiento de lenguaje natural, **lematizar** significa **convertir una palabra a su lema** para que todas las variantes existentes cuenten como la misma unidad léxica.

Palabra original	Lema	Explicación
caminando	caminar	Se elimina la conjugación del verbo.
caminé	caminar	Mismo verbo, distinta forma verbal.
mejores	bueno	Se reduce al adjetivo base.
estudiantes	estudiante	Se quita el plural.
fue	ser	El verbo “fue” (pretérito) se normaliza a su forma base “ser”.

☆ ***Lema:*** *(pt 2)*

👉 **Lematizar es más preciso (*que stemming, por ejemplo*)**, pero también más costoso computacionalmente, ya que requiere usar modelos (como *udpipe*) para poder realizar este procedimiento.





{udpipe}

udpipe es una librería de R desarrollada por la Universidad de Gröningen y otras instituciones europeas.

Su objetivo es hacer **análisis lingüístico completo** (*tokenización, lematización, etiquetado gramatical y análisis sintáctico*) **en múltiples idiomas**, sin depender de servicios externos ni conexión a internet haciendo uso de modelos lingüísticos pre-entrenados.

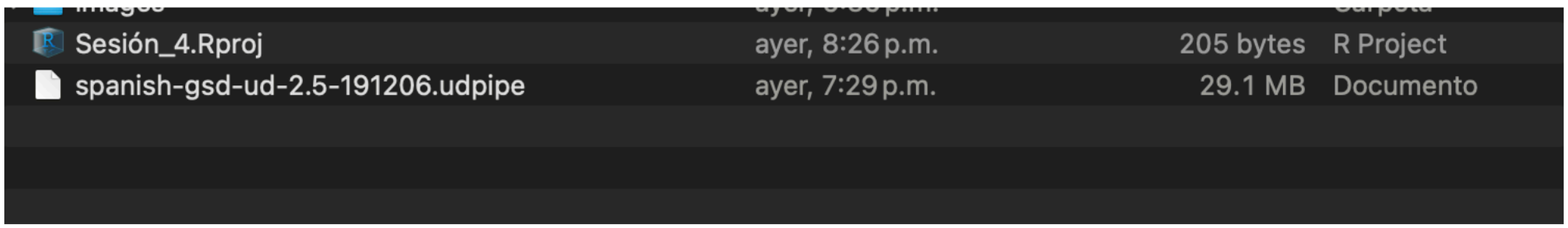
📦 *¿Qué hace udpipe_download_model()?*

Esa función **descarga un modelo de lenguaje entrenado** para el idioma que tú elijas, por ejemplo:

```
modelo <- udpipe_download_model(language =
“spanish")
```

spanish-gsd-ud-2.5-191206.udpipe

Este archivo contiene el **modelo estadístico de procesamiento del idioma español**, basado en el proyecto **Universal Dependencies (UD)**.



🧠 *¿Qué contiene el modelo?*

El modelo .*udpipe* incluye reglas y pesos entrenados para:

- **Tokenización** (dividir el texto en palabras)
- **Lematización** (encontrar el lema/base)
- **POS tagging** (etiquetar las categorías gramaticales: sustantivo, verbo, adjetivo, etc.)
- **Morfología** (género, número, persona, tiempo, modo…)
- **Parsing** (estructura de dependencias: sujeto-verbo-objeto)

Todo esto se aprende a partir de corpus anotados manualmente por lingüistas, siguiendo estándares internacionales.

Ejemplo de lematización con udpipe:

```r
library(udpipe)

# 1. Descargar modelo
modelo <- udpipe_download_model(language = "spanish")

# 2. Cargar el modelo
udmodel <- udpipe_load_model(modelo$file_model)

# 3. Analizar texto
texto <- "Los estudiantes estaban caminando al colegio."
anotado <- udpipe_annotate(udmodel, x = texto)
resultado <- as.data.frame(anotado)

# 4. Ver resultados
head(resultado[, c("token", "lemma", "upos", "feats")], 9)
```

Ejemplo de lematización con udpipe:

```r
library(udpipe)

# 1. Descargar modelo
modelo <- udpipe_download_model(language = "spanish")

# 2. Cargar el modelo
udmodel <- udpipe_load_model(modelo$file_model)

# 3. Analizar texto
texto <- "Los estudiantes estaban caminando al colegio."
anotado <- udpipe_annotate(udmodel, x = texto)
resultado <- as.data.frame(anotado)

# 4. Ver resultados
head(resultado[, c("token", "lemma", "upos", "feats")], 9)
```

```
      token      lemma  upos                                                    feats
        Los         el   DET    Definite=Def|Gender=Masc|Number=Plur|PronType=Art
estudiantes estudiante  NOUN                                          Number=Plur
    estaban      estar   AUX Mood=Ind|Number=Plur|Person=3|Tense=Imp|VerbForm=Fin
  caminando    caminar  VERB                                        VerbForm=Ger
         al       <NA>  <NA>                                                <NA>
          a          a   ADP                                                <NA>
         el         el   DET    Definite=Def|Gender=Masc|Number=Sing|PronType=Art
    colegio    colegio  NOUN                                  Gender=Masc|Number=Sing
          .          .  PUNCT                                                <NA>
```

Columna upos (Universal Part-of-speech tag)

Catálogo completo de los 17 valores posibles:

upos	Categoría	Ejemplos en español
ADJ	Adjetivo	grande, mexicano, nueva
ADP	Adposición (preposición)	de, en, por, con, desde
ADV	Adverbio	muy, siempre, ayer, no
AUX	Verbo auxiliar	*ha* comido, *está* trabajando, *puede* ir, *será*
CCONJ	Conjunción coordinante	y, o, pero, ni
DET	Determinante	el, la, un, este, mi, todos
INTJ	Interjección	hola, ay, bueno (como muletilla)
NOUN	Sustantivo común	ciudad, datos, trabajo
NUM	Numeral	tres, 2024, 15%
PART	Partícula	poco usada en español; en inglés *not*, *'s*
PRON	Pronombre	yo, ella, se, que (relativo), nadie
PROPN	Nombre propio	México, Juvenal, Tecnológico
PUNCT	Signo de puntuación	. , ; ¿ ?
SCONJ	Conjunción subordinante	que, porque, si, aunque, cuando
SYM	Símbolo	$, %, @, emojis
VERB	Verbo principal	comer, trabaja, analizaron
X	Otro / no clasificable	palabras extranjeras, errores tipográficos, fragmentos

Ejercicio 01

Realice los siguientes ejercicios:

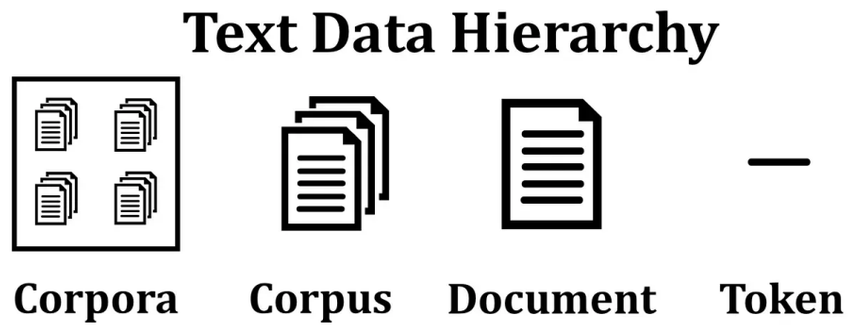
1) Descargue el modelo de *udpipe* necesario para lematizar las palabras utilizadas en español
2) A partir de este modelo, genere, a partir de las 10 mañaneras más recientes, un **objeto que tenga las palabras, los tokens, los lemmas, los stems, el upos (clasificación de la palabra) y los Feats (características de la palabra)**. Esto lo calcula la funcion *udpipe_annotate()*.
3) Cuente el total de **términos únicos** que hay en la columna de *tokens*, la columna de *lemma* y la columna de *stem*. *¿Cuál tiene menos términos? ¿Cuál usaría para saber cuales son las palabras más relevantes por mañanera? ¿Por qué?*

☆ TF-IDF

TF-IDF: (Term Frequency - Inverse Document Frequency)

Métrica para medir la **importancia de una palabra** tanto dentro de un documento como dentro de una colección de documentos (*corpus*).

El valor del TF-IDF **destaca las palabras que son frecuentes en un documento, pero raras en el resto del corpus.**



☆ TF-IDF (pt. 2)

Dada una palabra o término *t* en un documento *d*, el valor de TF-IDF se calcula con la siguiente fórmula:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Donde TF es igual a la frecuencia de veces que aparece la palabra t en el documento

$$TF(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Donde $f_{t,d}$ es la frecuencia de la palabra t en el documento d. Y donde el denominador es el número total de todas las palabras que aparecen en el documento d.

Ejemplo de cálculo de TF

Imagina un documento con 100 palabras y la palabra **“datos”** aparece **5 veces**:

Esto significa que la palabra *“datos”* representa el **5% del texto** de ese documento.

Por lo que TF = 0.05

$$TF(\text{"datos"}, d) = \frac{5}{100} = 0.05$$

☆ *TF-IDF (pt. 3)*

Y donde IDF se calcula con la siguiente fórmula:

$$IDF(t) = \log\left(\frac{N}{n_t}\right)$$

Donde **N** es el total de documentos en el corpus y $\mathbf{n_t}$ es el número de documentos donde el término aparece.

Si recordamos que el log(1) = 0, tenemos que **entre más documentos contengan la palabra *t*, el componente *IDF* se va haciendo más chiquito,** lo que hace que el TF-IDF total también tienda a cero.

☆ *TF-IDF (pt.4)*

🔷 Un valor alto de TF-IDF implica que:

- La palabra aparece **varias veces** en el documento (TF alto).
- Pero aparece **pocas veces o en pocos otros documentos** (IDF alto).

<u>En otras palabras:</u>

La palabra es **importante o distintiva dentro de ese documento** en comparación con el resto.

🔷 Un valor bajo de TF-IDF implica que:

- La palabra aparece **pocas veces** en el documento (TF bajo),

  o

- Aparece **en casi todos los documentos del corpus** (IDF bajo),
- **o ambas**.

<u>En otras palabras:</u>

La palabra **no es relevante para distinguir ese documento de los demás**.

Ejemplo de cálculo de TF-IDF

```r
# Tres textos:
t1 <- "My name is John. My best friend is Joe. We like tacos."
t2 <- "Two common best friend names are John and Joe."
t3 <- "Tacos are my favorite food. I eat them with my buddy Joe."

# Crear un data.frame con nuestros textos de ejemplo
df <- data.frame('text' = c(t1, t2, t3), 'ID' = c(1, 2, 3))

# Pipeline completo para calcular TF-IDF
df %>%
  unnest_tokens(output = "word", token = "words", input = text) %>%
  anti_join(stop_words) %>%
  count(ID, word, sort = TRUE) %>%
  bind_tf_idf(term = word, document = ID, n = n) %>%
  arrange(ID, -tf_idf)
```

Ejemplo de cálculo de TF-IDF

```r
# Tres textos:
t1 <- "My name is John. My best friend is Joe. We like tacos."
t2 <- "Two common best friend names are John and Joe."
t3 <- "Tacos are my favorite food. I eat them with my buddy Joe."

# Crear un data.frame con nuestros tex
df <- data.frame('text' = c(t1, t2, t3

# Pipeline completo para calcular TF-I
df %>%
  unnest_tokens(output = "word", token
  anti_join(stop_words) %>%
  count(ID, word, sort = TRUE) %>%
  bind_tf_idf(term = word, document =
  arrange(ID, -tf_idf)
```

```
Joining with `by = join_by(word)`
   ID     word n        tf       idf     tf_idf
1   1   friend 1 0.2500000 0.4054651 0.10136628
2   1     john 1 0.2500000 0.4054651 0.10136628
3   1    tacos 1 0.2500000 0.4054651 0.10136628
4   1      joe 1 0.2500000 0.0000000 0.00000000
5   2   common 1 0.2000000 1.0986123 0.21972246
6   2    names 1 0.2000000 1.0986123 0.21972246
7   2   friend 1 0.2000000 0.4054651 0.08109302
8   2     john 1 0.2000000 0.4054651 0.08109302
9   2      joe 1 0.2000000 0.0000000 0.00000000
10  3    buddy 1 0.1666667 1.0986123 0.18310205
11  3      eat 1 0.1666667 1.0986123 0.18310205
12  3 favorite 1 0.1666667 1.0986123 0.18310205
13  3     food 1 0.1666667 1.0986123 0.18310205
14  3    tacos 1 0.1666667 0.4054651 0.06757752
15  3      joe 1 0.1666667 0.0000000 0.00000000
```

Nótese que el TF-IDF es una medida por par palabra-documento. TF es una medida por palabra/término e IDF es también por palabra/documento.

Ejercicio 02

OBTENGA EL VALOR DE TF-IDF DE LAS MAÑANERAS

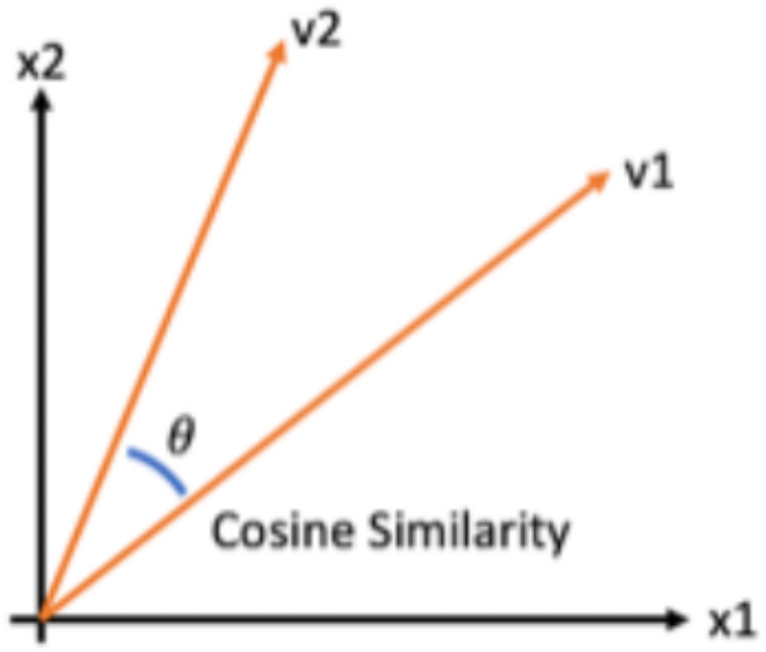
Dado el objeto que guarda las 10 mañaneras más recientes:

1) Calcule la matríz tf-idf para las 10 mañaneras más recientes
2) Filtre para quedarse con los 10 términos con el tf-idf más alto.
3) Compare las 10 palabras más frecuentes y las 10 con el tf-idf más alto. ¿Se parecen? ¿Cuál considera que es el mejor método para obtener los términos más relevantes?

☆ *Similitud del Coseno (Cosine Similarity)*

Medida fundamental para comparar documentos en el análisis de texto.

Esta métrica nos permite cuantificar qué tan similares son dos textos basándose en sus representaciones vectoriales.



Ejemplo de representación vectorial (BoW)

Dados los siguientes textos:

- `t1 <- c("Today will be an awesome day. The best day of the week")`
- `t2 <- c("Yesterday was an awesome day. Better than today.")`
- `t3 <- c("Tomorrow will be the best day. Better than yesterday and today.")`

Y el siguiente vector de palabras:

```
word_vector <- c("today", "awesome", "day", "best", "week", "yesterday", "better",
"tomorrow").
```

*** La representación BoW de t3 sería: [1, 0, 1, 1, 0, 1, 1, 1]**
*** La de t2 sería: [1,1,1,0,0,1,1,0]**
*** ¿Y la de t1?**

Similitud por pares con {widyr}

El paquete *widyr* proporciona una función muy útil para calcular similitudes entre todos los pares de documentos:

```
widyr::pairwise_similarity(tbl, item, feature, value, ...)
```

Parámetros:

- **tbl**: una tabla o tibble con los datos
- **item**: los elementos a comparar (artículos, tweets, etc.)
- **feature**: columna que describe la conexión entre los elementos (i.e., palabras)
- **value**: la columna de valores (i.e., “n” o “tf_idf”)

Similitud por pares con {widyr}

```
tf_idf_completa <- lematizacion |>
  anti_join(stopwords_es) |>
  group_by(doc_id, lemma) |>
  count() |>
  arrange(-n) |>
  filter(!is.na(lemma)) |>
  bind_tf_idf(term = lemma, document = doc_id, n = n)

tf_idf_completa |>
  ungroup() |>
  pairwise_similarity(item = doc_id,
                      feature = lemma,
                      value = tf_idf,
                      upper = FALSE) |>
  arrange(-similarity)
```

Primero obtenemos la matríz tf-idf completa

Y después ya le calculamos la similitud del coseno

Similitud por pares con {widyr}

```
tf_idf_completa <- lematizacion |>
  anti_join(stopwords_es) |>
  group_by(doc_id, lemma) |>
  count() |>
  arrange(-n) |>
  filter(!is.na(lemma)) |>
  bind_tf_idf(term = lemma, document = doc_id, n = n)

tf_idf_completa |>
  ungroup() |>
  pairwise_similarity(item = doc_id,
                      feature = lemma,
                      value = tf_idf,
                      upper = FALSE) |>
  arrange(-similarity)
```

Primero obtenemos la matríz tf-idf completa

Y después ya le calculamos la similitud del coseno

Similitud por pares con {widyr}

A mayor valor, mayor similaridad de contenidos

A tibble: 45 × 3

	item1	item2	similarity
	<chr>	<chr>	<dbl>
1	Mañanera del 09 de octubre de 2026	Mañanera del 02 de octubre de 2026	0.297
2	Mañanera del 06 de octubre de 2026	Mañanera del 29 de septiembre de 20…	0.273
3	Mañanera del 05 de octubre de 2026	Mañanera del 28 de septiembre de 20…	0.160
4	Mañanera del 05 de octubre de 2026	Mañanera del 09 de octubre de 2026	0.118
5	Mañanera del 29 de septiembre de 2026	Mañanera del 28 de septiembre de 20…	0.118
6	Mañanera del 09 de octubre de 2026	Mañanera del 28 de septiembre de 20…	0.113
7	Mañanera del 30 de septiembre de 2026	Mañanera del 07 de octubre de 2026	0.104
8	Mañanera del 01 de octubre de 2026	Mañanera del 29 de septiembre de 20…	0.101
9	Mañanera del 05 de octubre de 2026	Mañanera del 08 de octubre de 2026	0.0936
10	Mañanera del 05 de octubre de 2026	Mañanera del 02 de octubre de 2026	0.0907

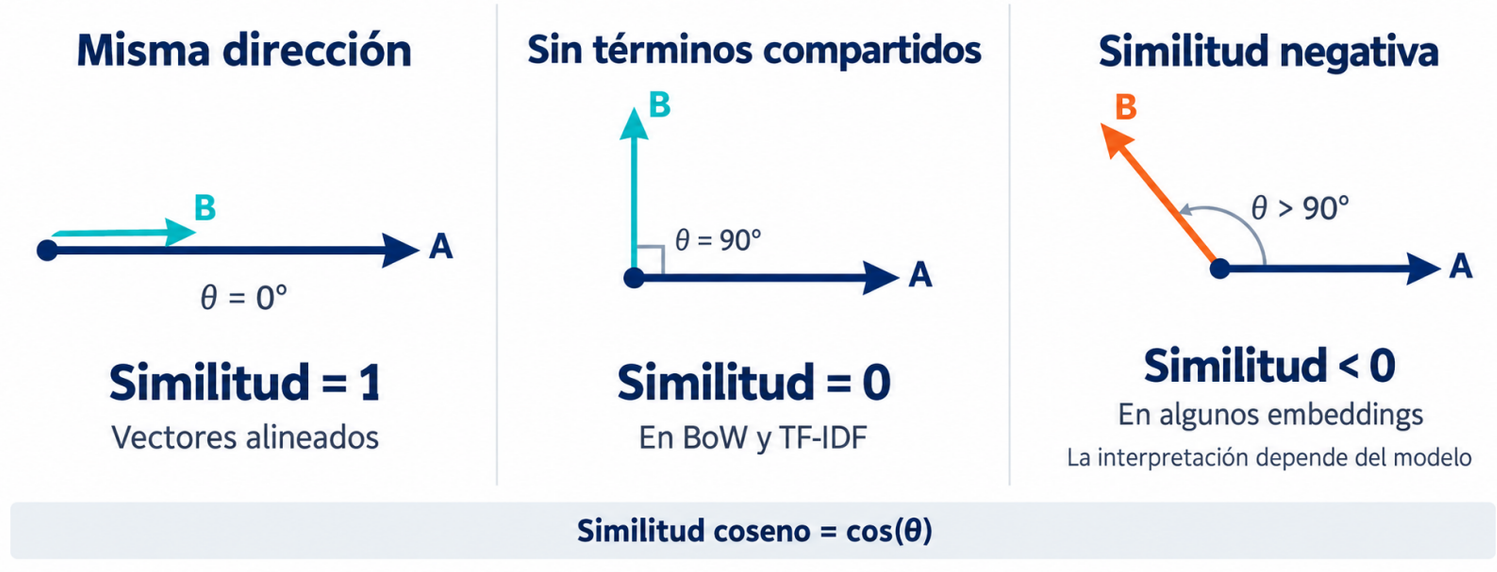
ℹ 35 more rows

ℹ Use `print(n = ...)` to see more rows

☆ *Similitud del Coseno (Cosine Similarity) pt.2*

Interpretación intuitiva

- Si el ángulo entre los vectores es **0°**, los textos apuntan en la misma dirección → similitud = **1**.
- Si el ángulo es **90°**, los textos no comparten términos relevantes → similitud = **0**.
- En algunos contextos (por ejemplo, embeddings), también puede haber **valores negativos**, indicando oposición semántica, aunque en el análisis clásico de texto (BoW, TF-IDF) esto no suele ocurrir.



☆ *Similitud del Coseno (Cosine Similarity) pt.3*

Casos de uso de la similaridad del coseno:

La similitud del coseno es una medida de **distancia** para comparar distintos elementos. Algunas de sus aplicaciones serían las siguientes:

Dominio	Aplicación típica	Ejemplo
Texto	Detección de duplicados	Plagio o respuestas similares
Texto	Clustering o clasificación	Agrupar por tema
Semántica	Comparar palabras	“salud” ≈ “bienestar”
Recomendaciones	Filtrado colaborativo	Usuarios con gustos similares
Deep Learning	Embeddings	Relación entre texto e imagen

Ejercicio 03

Dadas las 10 mañaneras más recientes:

1) Obtenga la similitud del coseno entre cada una de ellas, a partir del cálculo de TF-IDF, y
2) Determine cuales son las mañaneras más parecidas.